COMPSCI 701:
Creating Maintainable Software

Lecture 4.2: Code Quality Principles

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Admin
  - no preassigned groups for class exercise tomorrow
  - participation means in-class participation (without permission)
  - Assignment 2?
- Code Quality Principles — Rationale for programming conventions

Fundamental Question

- How do we determine what the consequences of following (or not) a given convention? $\Rightarrow$ what is its rationale
- How do we decide between competing conventions? $\Rightarrow$ what is its rationale
- When given a new convention, how to we decide whether to adopt it? $\Rightarrow$ what is its rationale
- To apply programming conventions to improve maintainability of code, we need to know the **rationale** for each convention

Rationalising "Code Style"

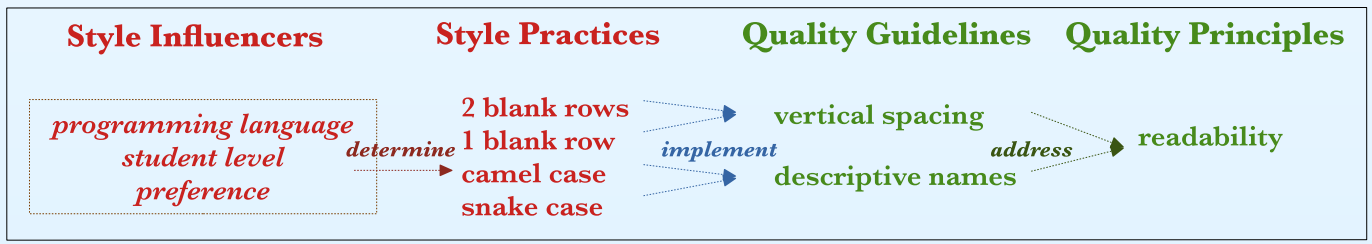
- 2018 — Provide support to High School Teachers (NCEA) to teach "good code style" How hard could it be. . . ? (with Andrew Luxton-Reilly)
- Reasonable progress made (especially after Diana Kirk joined the team), but:
  - ok for set of conventions for one language, but what about multiple conventions and multiple languages?
  - had to develop justifications for practically every convention (and then there are the inconsistent conventions!)
- Academic approach: create a model for explaining all conventions — at least for comprehensibility and alterability (modifiability)
- Basic idea: identify the principles that provide a foundation for explaining the validity of the conventions
- $\Rightarrow$ (then) Code Style Model — CSM; (now) Code Quality Principles — CQP

Methodology

- Talked to teachers to better understand what the problem is
- Looked at how others approach teaching “code style” (mostly at University level)
- Put on workshops for teachers
- Looked at how software quality and code style was discussed and organised in the research literature (e.g. Software Quality Models)
- Developed candidate set of principles (2024)
- Survey of University Educators (2025)
- Evaluating principles by considering all relevant conventions and whether and which principle explain them
- Refine and re-evaluate
- Refine and re-evaluate
- (present day)
- Refine and re-evaluate
- Refine and re-evaluate
- (etc)

Code Style != Code Quality

- mostly people talk about code style, but what we actually care about is code quality — what's the difference?
- quality describes an intrinsic property — style depends on context



Code Style Model/Code Quality Principles (CQP)

- **Clear Layout/Clear Presentation** Different elements are easy to recognise and distinguish and the relationships between them are apparent.
- **Explanatory Language** The rationale, intent and meaning of code is explicit.
- **Simple Constructs** Coding constructs are selected to minimise complexity for the intended reader.
- **Be Consistent** Elements that are similar in nature are presented and used in a similar way.
- **Avoid Duplication/Single Definition** Code duplication is avoided.
- **No Unused Content/Contributing Content** All elements that are introduced add immediate value.
- **Modular Structure** Related code is grouped together and dependencies between groups minimised.
- **Congruent Implementation** Implementation choices are consistent with the problem to be solved.

Clear Layout/Clear Presentation

Principle Different elements are easy to distinguish and the relationships between them are apparent.

Rationale Clear layout improves our shared understanding by making the individual elements easy to identify and signalling the elements the author considers to be related.

Guidelines

- Formatting (horizontal and vertical spacing, indentation, brackets and line length) should make distinct elements clearly visible.
- Placement of elements within a file should highlight the intended structure.

Explanatory Language

Principle The rationale, intent and meaning of code is explicit.

Rationale Being explicit in describing the purpose of the code elements helps us understand the author's intention, thus improving understandability.

Guidelines

- Comments and headers should describe rationale, intent and meaning and be clear and unambiguous to the intended reader.
- Names of elements (variables, functions, etc.) should be descriptive and contextually appropriate.
- Embedded literals that represent a specific concept should be extracted as named constants that explain the concept.

Simple Constructs

Principle Coding constructs are selected to minimise complexity for the intended reader.
Rationale Code that is perceived by the reader as simple is easier to understand.
Guidelines

- Nesting levels should be minimised.
- Flow of control should be easy to follow with few breaks and exceptions.
- The size and complexity of expressions should be appropriate for the intended reader.

Be Consistent

Principle Elements that are similar in nature are presented and used in a similar way.
Rationale Consistency leverages familiarity to reduce the mental effort required to understand the code.
Guidelines

- Documentation and notation should be used consistently throughout.
- When making decisions about code, be consistent in implementation choices.
- Code should adhere to the programming language idiom.

Avoid Duplication/Single Definition

Principle Code duplication is avoided.

Rationale Duplicated code (functions, literals) is difficult to change because there is a risk that not all items are changed.

Guidelines

- Duplicated code should be separated out into separate units, e.g., functions and classes.
- Embedded literals that represent a specific concept and are used multiple times should be abstracted as constants.

No Unused Content/Contributing Content

Principle All elements that are introduced are meaningfully used.
Rationale Unused code and documentation requires unnecessary mental effort.
Guidelines

- Unused code and documentation should be removed to reduce mental effort.
- Unreachable code should be removed.
- Statements that do not affect the logical flow of the program (for example, multiple breaks) should be removed.

Modular Structure

Principle Related code is grouped together and dependencies between groups minimised.
Rationale Placing related elements together makes code easier to understand. Reducing inter-connectedness means that isolated pieces can be more easily understood and can be modified independently.

Guidelines

- Modules should be created with clearly defined responsibilities and minimal dependence on other modules.
- Functions should aim to implement a single task and have a minimum number of parameters and side effects.
- Code should be organised such that the most related elements appear together.
- The scope of constants and variables should be minimised.

Congruent Implementation

Principle Implementation choices are consistent with the problem to be solved.
Rationale An implementation that reflects the problem is easier to change.
Guidelines

- Implementation choices, such as the structure used, are consistent with the nature of the problem to be solved.
- Identifiers and variables should not be reused within the same scope for different purposes.

Difficulties

- To be successful, CQP must provide the rationale for all (relevant) conventions — Many conventions have no rationale provided so we have to guess
- Getting the wording of: name of the principle, general rationale, specific guidelines; has been difficult at times
  - Avoid Duplication → Avoid Repetition → Single Definition
  - Clear Layout → Clear Presentation
  - Non-redundant Content → Contributing Content
  - Consistent Design — Guidelines
    - Design techniques should be applied consistently throughout (now “When making decisions about code, be consistent in implementation choices”)
    - Code design and constructs should adhere to the programming language idiom.

    — seem to have different scope
- Focus is on what the issue is, rather than how to address it. Changes may apply to more than one principle
- There are tradeoffs — improving with respect to one principle may reduce with respect to another.

Remaining Questions

- Is CQP finished? No!
- Does CQP makes sense? (Survey and some students labs suggest maybe?)
- Does CQP actually help? (To be determined)
- How to use CQP when teaching “code style” (code quality)?

Class Exercise

- Consider that the following:

```
 if x < y:
    print(1)
elif x >= y:
    print(2)
```

  can be replaced by:

```
if x < y:
    print(1)
else:
    print(2)
```

- Form teams of at least **TWO** people and post responses to the following:
  1. Why is the replacement “better”?
  2. Which CSQ (CSM) principle explains why?
  3. Consider the multiple uses of `result` from Discussion 2.1a. Which principle applies to this issue?

Key Points

- The Code Style Model (now renamed Code Quality Principles) is meant to provide a foundation for explaining why programming conventions (at least those applying to comprehensibility and alterability) improve code quality — provides rationale for programming conventions
- Mostly works reasonably well
- CSM/CQP is under active evaluation and development